

GUIA DE REFERENCIA TÉCNICA

Micro-framework Arquitectónico LC/NC para n8n

Trabajo de grado · Maestría en Gestión, Aplicación y Desarrollo de Software (MGADS) · UNAB · 2026

Autor: Elian Hernando Gil Sierra **Director:** Sebastian Roa Prada, PhD

1. El Problema

Las plataformas Low-Code / No-Code (LC/NC) como n8n, Zapier, Make o Power Automate han ganado adopción masiva en entornos empresariales. Sin estructura arquitectónica, generan deuda técnica crítica y sistémica documentada (Binzer et al., 2024; Viljoen et al., 2024).

Antipatrón frecuente	Impacto operativo
Secretos y API keys hardcoded en el JSON exportado	Riesgo de seguridad inmediato ante cualquier exportación del flujo
Sin mecanismo de retry ni idempotencia	Datos duplicados o perdidos ante fallos transitorios de red
Lógica de negocio acoplada a integraciones externas	Cambios simples impactan 5–8 nodos; alto costo de mantenimiento
Sin error workflow definido	Fallos silenciosos e irrecuperables; diagnóstico manual en la UI
Ausencia de log estructurado	MTTD promedio ~7 minutos; imposible automatizar alertas de fallo

2. La Propuesta: El Micro-Framework

Se diseñó un micro-framework arquitectónico para n8n basado en Clean Architecture (Martin, 2017) y prácticas DevSecOps. No modifica n8n ni requiere herramientas externas: es un conjunto de decisiones de diseño y reglas verificables aplicadas directamente al construir flujos.

2.1 Metamodelo: Cuatro Etapas Funcionales (E1–E4)

Flujo de ejecución:

webhook → E1 Validación → E2 Dominio → E3 Adaptadores → E4 Salida → respond

Si falla cualquier etapa → Error Workflow captura el fallo y preserva el payload en dead-letter PostgreSQL

Eta	Responsabilidad	Restricción clave
E1 · VALIDACIÓN	Valida el payload contra schema estricto. Verifica autenticación y autorización.	No ejecuta lógica de negocio ni llama APIs externas.
E2 · DOMINIO	Ejecuta lógica de negocio pura: cálculos, clasificaciones, enrutamiento.	No llama BD ni APIs. Sin efectos secundarios externos.
E3 · ADAPTADORES	Interactúa con servicios externos: BD, APIs de terceros, colas.	Una integración por nodo. Implementa idempotencia obligatoria.
E4 · SALIDA	Produce la respuesta final: notificaciones, webhooks, registros de estado.	Último eslabón. Routing diferenciado por criticidad.

2.2 Las 10 Reglas Obligatorias (REG-001 a REG-010)

Criterio binario verificable automáticamente sobre el JSON exportado mediante el validador estático validar-flujos.mjs.

ID	Nombre	Criterio verificable
REG-001	Gestión de secretos	Sin tokens ni contraseñas literales en el JSON. Usar \$credentials.xxx
REG-002	Orquestación centralizada	Un único flujo orquestador por proceso. Subflujos invocados desde él.
REG-003	Error workflow obligatorio	Captura fallos. Preserva payload en dead-letter PostgreSQL.
REG-004	Retry con backoff	Reintentos con espera creciente en todas las llamadas HTTP salientes.
REG-005	Idempotencia	Idempotency-Key en HTTP / ON CONFLICT en BD en operaciones mutantes.

REG-006	Log estructurado JSON	Nodo de log al inicio y cierre de cada etapa: stage, timestamp, status.
REG-007	Validación de entrada en E1	Schema estricto antes de cualquier procesamiento. HTTP 400 ante inválidos.
REG-008	Dominio aislado en E2	E2 no realiza llamadas a BD ni APIs externas directamente.
REG-009	Contratos HTTP correctos	HTTP 401 ante token inválido · 400 ante payload malformado · 200 solo ante éxito real.
REG-010	Separación de integraciones	En E3, una integración externa por nodo. Sin mezcla de responsabilidades.

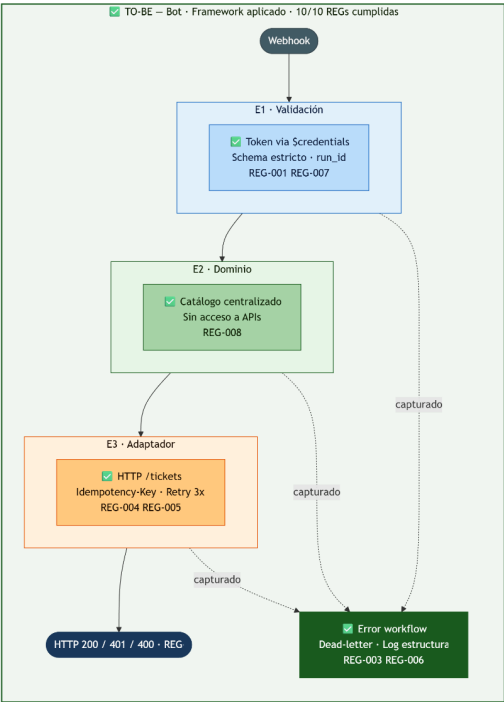
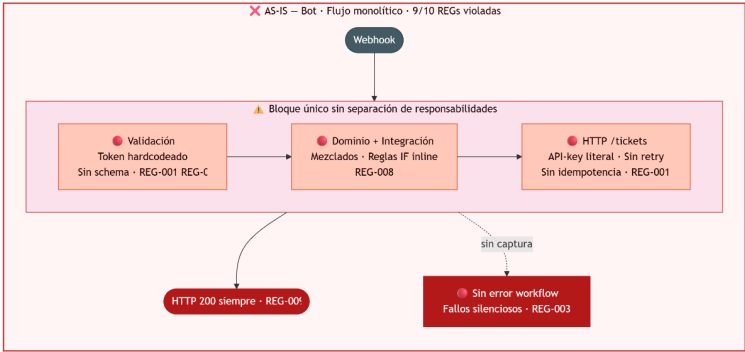
3. Casos de Estudio Evaluados

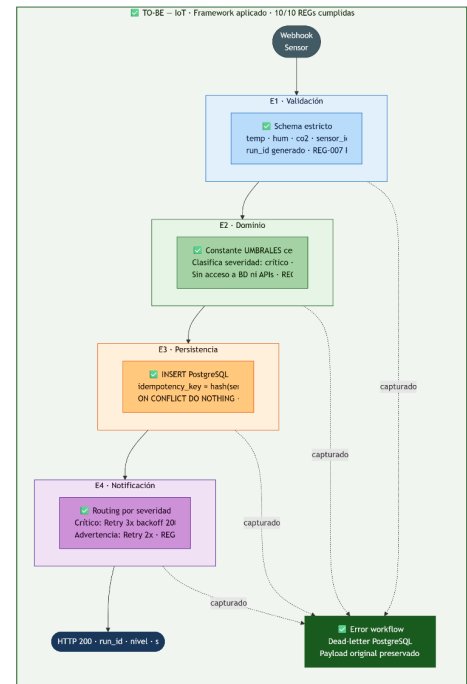
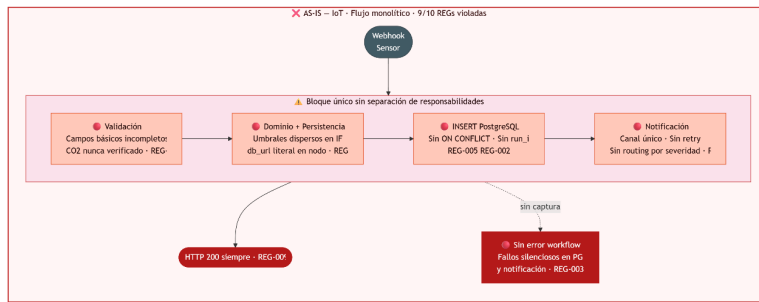
El framework se aplicó en dos casos representativos y ortogonales de uso real en n8n, cubriendo patrones síncronos y asíncronos de integración empresarial.

Escala total: 8.000 corridas · 12 Change Requests · 12 escenarios ATAM · 19 ADRs

CASO BOT — Sistema de Soporte por Mensajería	CASO IoT — Pipeline de Ingesta de Sensores
Tipo: Flujo síncrono · Recibe webhooks de mensajes, aplica reglas de prioridad y crea tickets en servicio externo.	Tipo: Flujo asíncrono · Recibe lecturas de sensores, valida, persiste en PostgreSQL y notifica según nivel de criticidad.
As-is: 16 nodos · token hardcodeado · sin retry · sin error workflow · 9/10 REGs violadas	As-is: 14 nodos · sin idempotencia · sin error workflow · credenciales expuestas · 9/10 REGs violadas
✓ To-be: Orquestador + E2 + E3 · 10/10 REGs cumplidas	✓ To-be: Orquestador + E1 + E2 + E3 + E4 + error handler · 10/10 REGs cumplidas

Los siguientes diagramas ilustran la transformación arquitectónica aplicada en cada caso. El diagrama as-is muestra la estructura monolítica original con los antipatrones activos; el diagrama to-be muestra la separación en etapas funcionales con los mecanismos de resiliencia incorporados. Las referencias REG identifican la regla del framework cumplida o violada en cada componente.





4. Resultados Cuantitativos: As-is vs To-be

Las métricas se midieron sobre 8.000 corridas controladas distribuidas en cuatro sets de entrada por caso (válidos, inválidos, reintentos, tokens incorrectos), ejecutadas bajo condiciones equivalentes en las versiones as-is y to-be. Los 12 Change Requests se aplicaron manualmente registrando nodos tocados, dependencias afectadas, intentos y tiempo por CR. Las métricas de seguridad se obtienen del validador estático validar-flujos.mjs, que analiza el JSON exportado de cada flujo y reporta violaciones de REG-001 a REG-010. El MTTD se midió calculando el tiempo desde la emisión del log de fallo hasta la identificación del campo stage en el log estructurado.

Métrica	Caso	As-is	To-be	Mejora	Meta
Nodos modificados / CR	Bot	5.3	1.0	-81%	✓ CUMPLIDA (meta ≥20%)
Nodos modificados / CR	IoT	4.3	0.7	-84%	✓ CUMPLIDA (meta ≥20%)
Tiempo por CR (min)	Bot	32.7	6.7	-79%	—
Tiempo por CR (min)	IoT	28.0	5.2	-81%	—
Tasa de fallos en ejecución	Bot	9%	6%	-33%	✓ CUMPLIDA (meta ≥30%)
MTTD diagnóstico de fallo	Bot	~7 min	14 s	-97%	✓ CUMPLIDA (meta ≤60 s)
Secretos literales en JSON	Bot	4	0	-100%	✓ CUMPLIDA (meta = 0)
Secretos literales en JSON	IoT	varios	0	-100%	✓ CUMPLIDA (meta = 0)
Cumplimiento checklist REGs	Bot+IoT	10%	100%	+90 pp	✓ CUMPLIDA (meta ≥90%)

5. Trade-offs Cuantificados

Los trade-offs se identificaron por inspección de los ADRs y se cuantificaron sobre los mismos run-logs de las 8.000 corridas. La latencia adicional de TP-GLOBAL-01 se mide en percentil 50 (p50) para excluir outliers de retry; la magnitud es real y deliberada: el proyecto prioriza mantenibilidad sobre rendimiento, decisión documentada en ADR-001. Ninguno de estos trade-offs impacta los escenarios top-K del ATAM porque ninguno evalúa eficiencia como atributo principal —la latencia se reporta como costo contextual, no como regresión.

ID	Trade-off	Atributos en tensión	Magnitud medida
TP-GLOB AL-01	Modularización vs Latencia	Mantenibilidad ++ vs Rendimiento --	Bot: +9% p50 · IoT: +119% a +192% p50 (4 subflujos secuenciales)
TP-GLOB AL-02	Validación estricta vs Flexibilidad	Corrección funcional ++ vs Flexibilidad evolutiva --	E1 rechaza cualquier payload que no cumpla el schema exacto definido.
TP-IOT-01	Routing diferenciado vs Latencia	Resiliencia canal crítico + vs Latencia base --	+10.8 ms p50 en canal crítico vs canal de advertencia (ADR-IoT-004).

El trade-off más significativo es la latencia adicional por modularización en IoT (+119%—+192% p50): costo deliberado asumido en favor de la mantenibilidad, documentado en ADR-001.

6. Hallazgos ATAM — Síntesis

La evaluación ATAM analiza arquitecturas de software respondiendo una pregunta práctica: ¿qué tan bien soporta el diseño situaciones reales de cambio, fallo o uso? Para este proyecto se definieron 12 escenarios concretos 6 por caso de estudio que simulan situaciones reales a las que estaría expuesto cada flujo. Cada escenario se evaluó sobre el diseño original (as-is) y el rediseño con el framework (to-be) usando una escala de 1 a 5, donde 1 significa que el diseño no soporta la situación y 5 que la soporta con evidencia verificable.

Atributo ISO 25010	Escenarios evaluados	¿Qué se evaluó?	As-is	To-be	Delta
Mantenibilidad	BOT-Q1: Modificabilidad de reglas	Si cambia una regla de negocio, ¿cuántos nodos hay que tocar?	2	5	+3.0
	BOT-Q2: Cambio de proveedor	Si se cambia el proveedor de tickets, ¿el cambio está aislado?	2	5	+3.0
	IOT-Q1: Ajuste de umbrales	Si cambia un umbral de temperatura, ¿el cambio está aislado?	2	5	+3.0
	IOT-Q2: Cambio de canal de alerta	Si cambia el canal de notificación, ¿el cambio está aislado?	2	5	+3.0
Fiabilidad	BOT-Q4: Idempotencia ante reintentos	Si el mismo mensaje llega dos veces, ¿se generan duplicados?	2	4	+2.0
	IOT-Q3: Idempotencia en persistencia	Si la misma lectura llega dos veces, ¿se persiste duplicada?	1	5	+4.0
	IOT-Q4: Tolerancia a fallos	Si el canal de notificación falla, ¿el sistema se recupera sin perder el dato?	1	4	+3.0
	IOT-Q5: Urgencia diferenciada	Si llegan lecturas críticas y de advertencia mezcladas, ¿se procesan con la urgencia correcta?	1	4	+3.0
Seguridad	BOT-Q3: Confidencialidad de credenciales	¿El flujo expone tokens si se exporta el archivo?	1	5	+4.0
	IOT-Q6: Confidencialidad de credenciales BD	¿El flujo expone la contraseña de la base de datos si se exporta el archivo?	1	5	+4.0
Operabilidad	BOT-Q5: Diagnóstico de fallos (MTTD ~14 s vs ~7 min)	Si ocurre un fallo, ¿en cuánto tiempo se identifica qué y dónde?	1	5	+4.0
Adec. Funcional	BOT-Q6: Corrección de contratos HTTP (401/400/200)	¿El flujo responde con el código HTTP correcto ante entradas inválidas?	2	5	+3.0

Clasificación de hallazgos por categoría ATAM

Sensitivity Points (SP) 3 identificados	Tradeoff Points (TP) 3 identificados	Risks abiertos (R) 4 identificados	Non-Risks (NR) 5 identificados
SP-BOT-01: Idempotencia depende de que el servicio externo respete Idempotency-Key. SP-BOT-02: MTTD depende de la estructura del campo stage en el log JSON. SP-IOT-01: Canal del error handler coincide con canal de E4.	TP-GLOBAL-01: Modularización vs latencia. TP-GLOBAL-02: Validación estricta vs flexibilidad evolutiva. TP-IOT-01: Routing diferenciado vs latencia nominal.	R-BOT-01: Rotación manual de tokens. R-IOT-01: Dead-letter puede fallar si E4 está caído. R-GLOBAL-01: Logs stdout efímeros sin sistema centralizado. R-GLOBAL-02: Idempotencia depende de APIs de terceros.	NR-IOT-01: E3 independiente de E4; dato persiste aunque falle notificación. NR-IOT-02: ON CONFLICT previene duplicidad ante reintentos. NR-BOT-01: E1 previene side-effects de payloads malformados. NR-BOT-02: HTTP 401/400 correctos en 100% de sets de prueba. NR-GLOBAL-01: Validador estático previene regresiones de seguridad.

7. Síntesis y Consideraciones

Los resultados reflejan una evaluación sistemática y trazable sobre dos casos representativos, con evidencia cuantitativa de **8.000 corridas** y evidencia arquitectónica de **12 escenarios ATAM y 19 ADRs documentados**. En todos los indicadores medidos el diseño to-be supera al as-is, sin regresión en ningún escenario.

Los resultados no son absolutos: tres sensitivity points señalan dependencias externas que el framework no puede controlar por sí solo. El trade-off de latencia en IoT (+119%—+192% p50) es un costo deliberado asumido en favor de la mantenibilidad.

Un framework que mejora la mantenibilidad en más del 80%, elimina el 100% de los secretos expuestos y reduce el MTTD de minutos a segundos —asumiendo un overhead de latencia controlado y documentado— representa una mejora neta verificable para el tipo de soluciones LC/NC que aborda.

Los scores ATAM corresponden al análisis del autor. La función del panel de expertos es confirmar, cuestionar o enriquecer estas clasificaciones con criterio profesional independiente.